

Extracción de características relacionales

En este notebook, utilizaremos NetworkX para calcular las métricas predictivas basadas en la teoría de grafos. Tenemos que extraer obligatoriamente información de centralidad, coeficiente de clustering y detección de comunidades.

Introducción

En esta tarea construiremos características relacionales a partir de la red de citas del dataset Cora. A diferencia de las propiedades nativas de cada artículo, como su vector de 1433 palabras, las características relacionales describen la posición y el papel de cada nodo dentro del grafo.

Para ello trabajaremos con el grafo no dirigido obtenido en la Tarea 1.3 y calcularemos métricas basadas en teoría de grafos que puedan aportar capacidad predictiva adicional. En particular, analizaremos medidas de centralidad, el coeficiente de clustering y la estructura de comunidades presentes en la red.

El objetivo es complementar la información atributiva de los artículos con información estructural, de modo que después pueda utilizarse en tareas de aprendizaje automático relacional, como clasificación de nodos o análisis de patrones de conectividad.

```
In [1]: from pathlib import Path

import pandas as pd
import networkx as nx

# Directorio base del proyecto
BASE_DIR = Path.cwd().parent

# Rutas al dataset
data_path = BASE_DIR / "data" / "cora"
path_cora_content = data_path / "cora.content"
path_cora_cites = data_path / "cora.cites"

# Cargar nodos, características y etiquetas
cora_content = pd.read_csv(path_cora_content, sep="\t", header=None)

paper_ids = cora_content.iloc[:, 0].astype(str)
features = cora_content.iloc[:, 1:-1]
labels = cora_content.iloc[:, -1]

df_nodes = pd.DataFrame({
    "paper_id": paper_ids,
    "label": labels
```

```

})

# Cargar citas (aristas)
cora_cites = pd.read_csv(
    path_cora_cites,
    sep="\t",
    header=None,
    names=["cited_paper_id", "citing_paper_id"]
)

cora_cites["cited_paper_id"] = cora_cites["cited_paper_id"].astype(str)
cora_cites["citing_paper_id"] = cora_cites["citing_paper_id"].astype(str)

# Construir el grafo no dirigido
G = nx.from_pandas_edgelist(
    cora_cites,
    source="cited_paper_id",
    target="citing_paper_id"
)

# Asegurar que todos los nodos de cora.content estén presentes en el grafo
G.add_nodes_from(paper_ids)

print(f"Número de nodos en el grafo: {G.number_of_nodes()}")
print(f"Número de aristas en el grafo: {G.number_of_edges()}")
print(f"Dimensión de la matriz de características: {features.shape}")
print(f"Número de clases: {labels.nunique()}")

```

Número de nodos en el grafo: 2708
 Número de aristas en el grafo: 5278
 Dimensión de la matriz de características: (2708, 1433)
 Número de clases: 7

Carga de datos y reconstrucción del grafo

En esta sección recuperamos la información generada a partir del dataset Cora para disponer del grafo de citas y de las características nativas de cada artículo. Con estos elementos construiremos la base necesaria para calcular métricas relacionales como centralidad, clustering y comunidades.

```

In [3]: import networkx as nx
import pandas as pd
from networkx.algorithms import community

# --- 1. MÉTRICAS DE CENTRALIDAD ---
# Calculamos la centralidad de grado y la centralidad de intermediación (bet
# Puedes calcular otras como nx.closeness centrality(G) si lo deseas
dict_degree_cent = nx.degree_centrality(G)
dict_betweenness_cent = nx.betweenness_centrality(G)

# --- 2. COEFICIENTE DE CLUSTERING ---
# Mide la tendencia de los nodos a agruparse en triángulos
dict_clustering = nx.clustering(G)

```

```
# --- 3. DETECCIÓN DE COMUNIDADES ---
# El documento sugiere métodos como Louvain, Girvan-Newman o Walktrap.
# Usaremos Louvain, que es muy eficiente y está integrado en NetworkX.
comunidades_louvain = community.louvain_communities(G)

# Convertimos la lista de comunidades en un diccionario {nodo: id_comunidad}
dict_comunidades = {}
for id_comunidad, set_nodos in enumerate(comunidades_louvain):
    for nodo in set_nodos:
        dict_comunidades[nodo] = id_comunidad

# --- 4. CONSOLIDACIÓN EN UN DATAFRAME ---
# Unimos todas las características extraídas en un DataFrame usando el ID de
df_relacional = pd.DataFrame({
    'centralidad_grado': pd.Series(dict_degree_cent),
    'centralidad_betweenness': pd.Series(dict_betweenness_cent),
    'coeficiente_clustering': pd.Series(dict_clustering),
    'comunidad_louvain': pd.Series(dict_comunidades)
})

# Renombramos el índice para que coincida con EDA previo
df_relacional.index.name = 'paper_id'

# Mostramos los primeros 5 nodos con sus nuevas características relacionales
df_relacional.head()
```

Out[3]:

	centralidad_grado	centralidad_betweenness	coeficiente_clustering	comunidad_louvain
1000012	0.001847	0.000548	0.200000	1
100197	0.001478	0.002665	0.166667	1
100701	0.000369	0.000000	0.000000	1
100935	0.001108	0.001105	0.000000	1
100961	0.001108	0.000021	0.666667	1

paper_id	centralidad_grado	centralidad_betweenness	coeficiente_clustering	comunidad_louvain
1000012	0.001847	0.000548	0.200000	1
100197	0.001478	0.002665	0.166667	1
100701	0.000369	0.000000	0.000000	1
100935	0.001108	0.001105	0.000000	1
100961	0.001108	0.000021	0.666667	1

¿Qué hace este código exactamente?

- 1. **Centralidad:**
 - `degree_centrality`: mide simplemente la cantidad de conexiones proporcionales
 - `betweenness_centrality`: mide cuántas veces un artículo hace de "puente" en el camino más corto entre otros dos.
- 2. **Clustering:** Calcula la densidad de triángulos locales para cada publicación.
- 3. **Comunidades (Louvain):** Agrupa la red en clústeres fuertemente conectados. Le asigna a cada publicación un número entero (`id_comunidad`) que representa el grupo temático subyacente que el algoritmo ha encontrado solo mirando los enlaces.

Al finalizar tenemos el DataFrame `df_relacional` listo.